

STYLEWISE — ARQUITETURA COMPLETA DO APLICATIVO DE MODA COM IA

ÍNDICE

1. [Análise do Conceito](#)
 2. [Nome e Posicionamento](#)
 3. [Estrutura Completa](#)
 4. [Mapa de Navegação](#)
 5. [Principais Telas](#)
 6. [Fluxo de Usuário](#)
 7. [Design System](#)
 8. [Arquitetura Frontend](#)
 9. [Arquitetura Backend](#)
 10. [Banco de Dados](#)
 11. [Sistema de IA](#)
 12. [Segurança e Privacidade](#)
 13. [Estratégia de MVP](#)
 14. [Tecnologias](#)
-

1. ANÁLISE DO CONCEITO

Proposta de Valor

STYLEWISE é um personal stylist digital baseado em IA que resolve três problemas principais:

1. **Indecisão diária:** “O que vestir hoje?”
2. **Subutilização do guarda-roupa:** Pessoas possuem roupas que raramente usam
3. **Desconexão com tendências:** Dificuldade em acompanhar moda sem gastar muito

Diferenciais Competitivos

- **Prioriza o guarda-roupa existente** (não é uma plataforma de vendas)
- **IA que aprende progressivamente** com as escolhas do usuário
- **Radar de Moda inteligente** que cruza tendências com o que você já possui
- **Análises explicativas** (não apenas “combina/não combina”)
- **Identidade visual premium** (revista de moda + tecnologia)

Público-Alvo

- **Primário:** Pessoas de 25-45 anos interessadas em moda, que querem otimizar seu guarda-roupa
 - **Secundário:** Entusiastas de moda que querem acompanhar tendências de forma inteligente
 - **Terciário:** Pessoas que têm dificuldade em combinar roupas e buscam orientação
-

2. NOME E POSICIONAMENTO

Nome do Aplicativo

STYLEWISE

“Style” (estilo) + “Wise” (sábio/inteligente)

Tagline

“Seu estilo. Sua inteligência.”

Posicionamento

Um personal stylist com inteligência artificial que entende seu guarda-roupa, aprende suas preferências e conecta você com as tendências de moda de forma inteligente e econômica.

3. ESTRUTURA COMPLETA

Módulos Principais

STYLEWISE

- 1. AUTENTICAÇÃO
 - Login
 - Cadastro
 - Recuperação de senha
- 2. ONBOARDING
 - Boas-vindas
 - Configuração de perfil
 - Tutorial rápido
- 3. HOME
 - Saudação personalizada
 - Acesso rápido às funções principais
 - Sugestões do dia
- 4. GUARDA-ROUPA DIGITAL
 - Galeria de peças
 - Adicionar peça (foto + IA)
 - Editar peça
 - Filtros e busca
 - Organização automática por categoria
- 5. ANÁLISE DE LOOK
 - Fotografar look atual
 - Análise da IA
 - Sugestões de melhoria
 - Alternativas do guarda-roupa
- 6. MONTE MEU LOOK
 - Seleção de peça base
 - Configuração (ocasião, clima, estilo)
 - Geração de combinações
 - Visualização de looks
 - Salvar favoritos
- 7. TENHO UMA OCASIÃO
 - Descrição da ocasião
 - Configurações (data, horário, clima)
 - Sugestões personalizadas
 - Salvar look
- 8. RADAR DE MODA
 - Tendências atuais
 - Filtros (categoria, status)
 - Detalhes da tendência
 - Gráficos de evolução
 - "Está em Alta + Combina Comigo?"
- 9. FAVORITOS
 - Looks salvos
 - Detalhes do look
 - Reutilizar look
- 10. HISTÓRICO
 - Looks analisados
 - Combinações geradas
 - Avaliações
- 11. PERFIL
 - Dados pessoais

- Preferências de estilo
 - Configurações
 - Privacidade
-
- 12. CONFIGURAÇÕES
 - Notificações
 - Clima (localização)
 - Privacidade
 - Sobre

4. MAPA DE NAVEGAÇÃO

Navegação Principal (Bottom Navigation)

-  Início
- |
-  Guarda-roupa
- |
-  Criar
- |
-  Radar
- |
-  Favoritos
- |

Hierarquia de Navegação

INÍCIO

- Analisar meu look → Análise de Look
- Montar meu look → Monte Meu Look
- Meu guarda-roupa → Guarda-roupa Digital
- Radar de Moda → Radar

GUARDA-ROUPA

- Ver todas as peças
- Adicionar peça
- Filtrar por categoria
- Buscar peça

CRIAR (Menu)

- Monte meu look
- Tenho uma ocasião
- Analisar meu look

RADAR

- Tendências atuais
- Está em Alta + Combina Comigo?
- Tendências que combinam com meu guarda-roupa
- Filtros

FAVORITOS

- Looks salvos
- Detalhes do look

5. PRINCIPAIS TELAS

5.1 Tela Inicial (Splash)

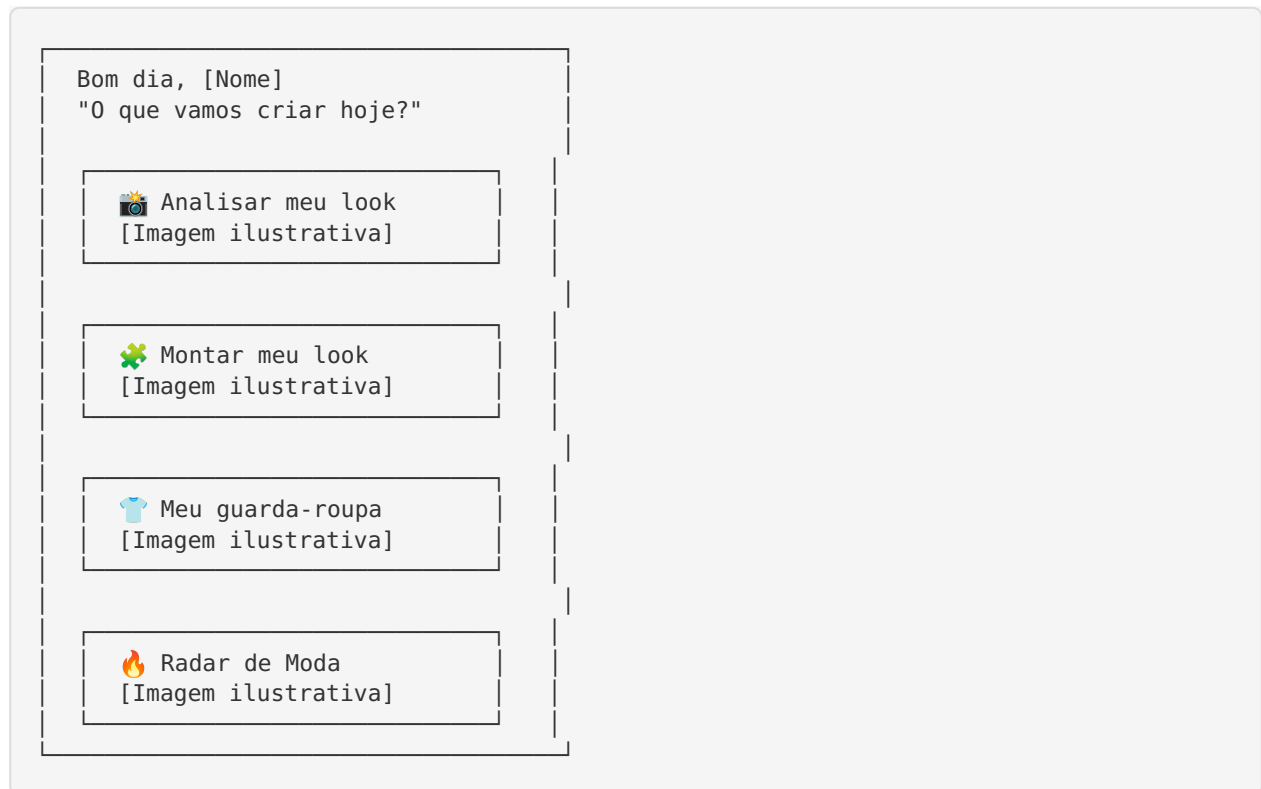
Elementos:

- Fundo: #F7F5F2

- Logo centralizada
 - Nome: **STYLEWISE** (preto #111111)
 - Detalhe dourado discreto na marca
 - Tagline: "Seu estilo. Sua inteligência."
 - Botão: "Começar" (fundo preto, texto branco)
-

5.2 Home

Layout:

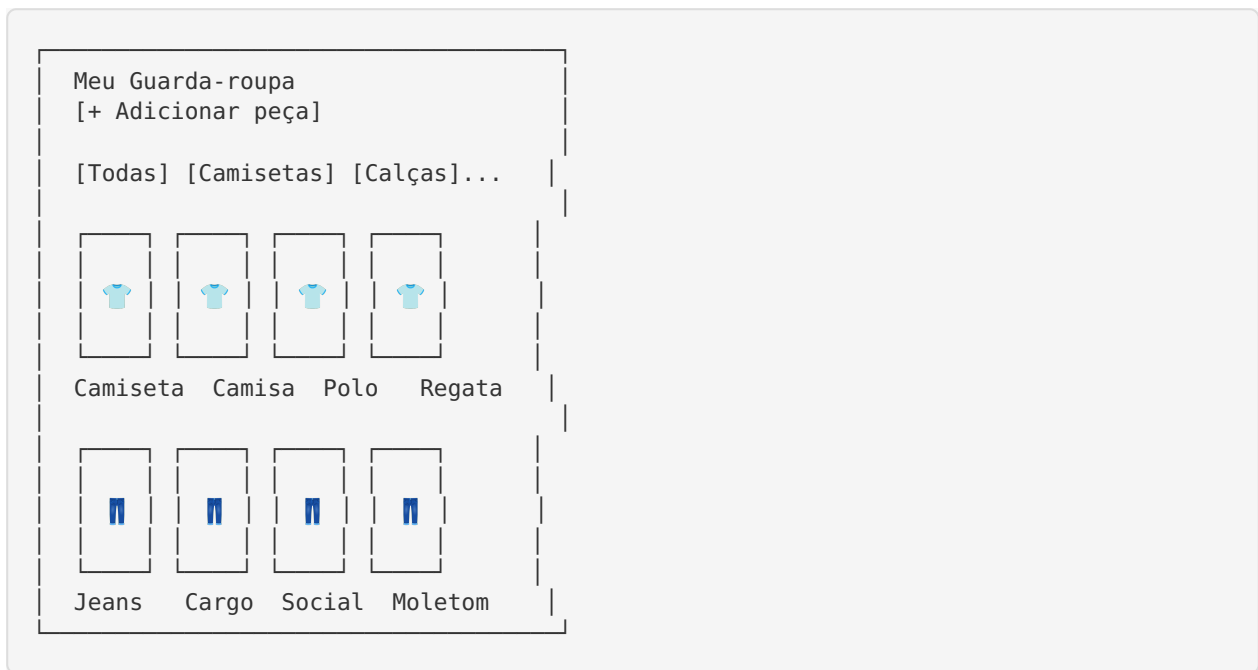


Características:

- Cards grandes com imagens
 - Espaçamento generoso
 - Fundo off-white
 - Cards brancos com sombra suave
-

5.3 Guarda-roupa Digital

Layout:



Funcionalidades:

- Grid de fotos das peças
- Filtros por categoria
- Busca inteligente
- Adicionar peça (câmera ou galeria)
- Editar/excluir peça

5.4 Adicionar Peça

Fluxo:

1. Captura da foto

- Câmera ou galeria
- Preview da imagem

2. Análise automática da IA

- Loading elegante
- “Analisando sua peça...”

3. Resultado da análise

[Foto da peça]
Categoria: Camiseta
Cor: Preta
Padrão: Lisa
Estilo: Casual
[Editar] [Salvar]

4. Confirmação

- Usuário pode editar
- Salvar no guarda-roupa

5.5 Análise de Look

Layout:

Análise do Look

[Foto do usuário]

Análise da IA

Combinação

8/10

Cores

10/10

Estilo

7/10

✓ 0 que funciona

• Cores neutras equilibradas

• Tênis branco cria contraste

• Visual casual coerente

⚡ 0 que pode melhorar

• Adicionar um acessório

💡 Sugestão da IA

"Experimente adicionar um relógio ou pulseira para finalizar."

[Ver alternativas do guarda-roupa]

5.6 Monte Meu Look

Layout:

Monte seu look

Peça selecionada

[Camiseta preta]

Ocasão

[Casual ▼]

Estilo

[Streetwear ▼]

Clima

[☀️ 25°C - Ensolarado]

[🌟 Criar combinações]

Resultado:

Combinações sugeridas

Look 1

[Visualização do look]

- Camiseta preta
- Calça jeans
- Tênis branco

"Visual casual equilibrado com contraste no calçado"

[👍] [👎] [❤️ Salvar]

Look 2

[Visualização do look]

...

5.7 Radar de Moda

Layout (fundo preto):

 Radar de Moda

[Todas] [Crescendo] [Consolidada]

 Jaqueta Oversized
[Imagem da tendência]

Tendência crescendo
Categoria: Jaquetas
[Ver detalhes]

 Tênis Chunky
[Imagem da tendência]

Tendência consolidada
Categoria: Calçados
[Ver detalhes]

Detalhes da Tendência:

 Jaqueta Oversized

[Imagem grande]

Status: Tendência crescendo
Categoria: Jaquetas

Evolução
[Gráfico minimalista]

Fontes utilizadas

- Google Trends
- Fashion API

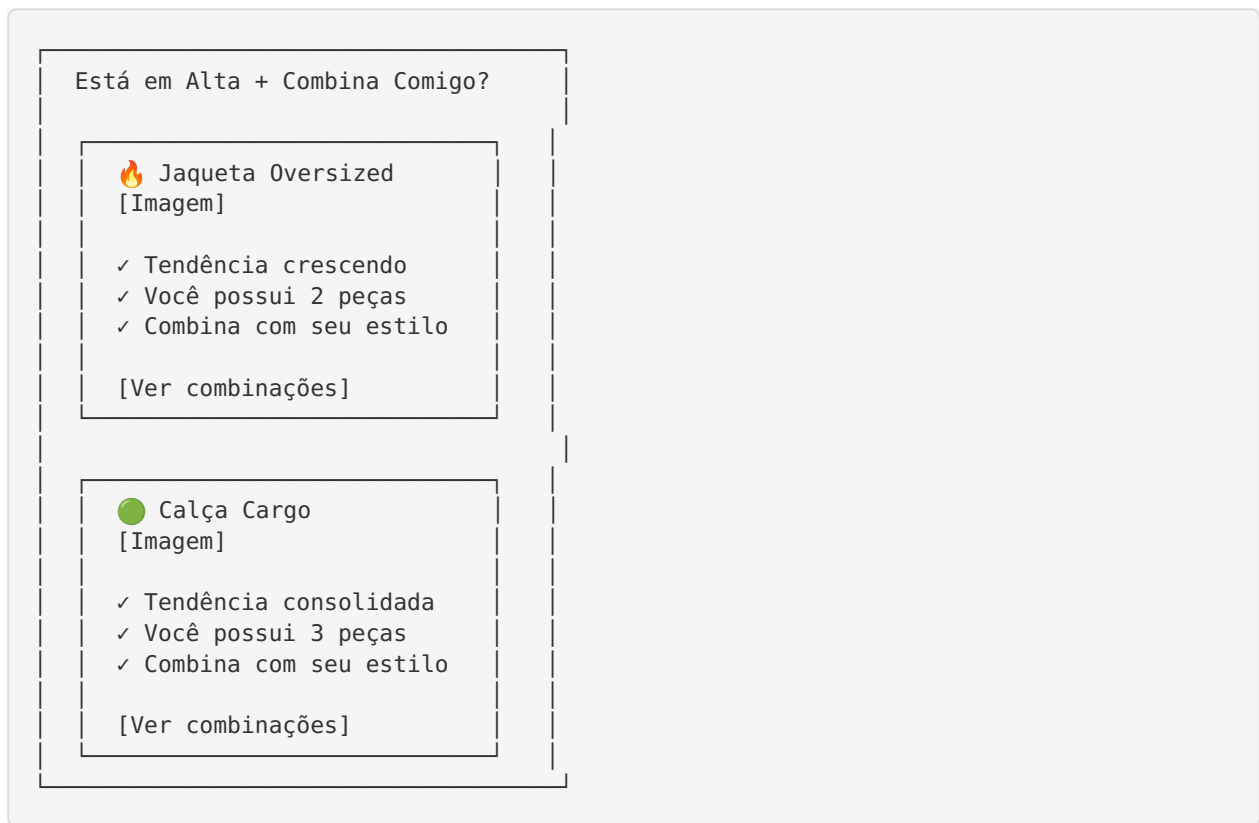
Última atualização: 20/09/2026

 Você possui 2 peças relacionadas

[Ver combinações]

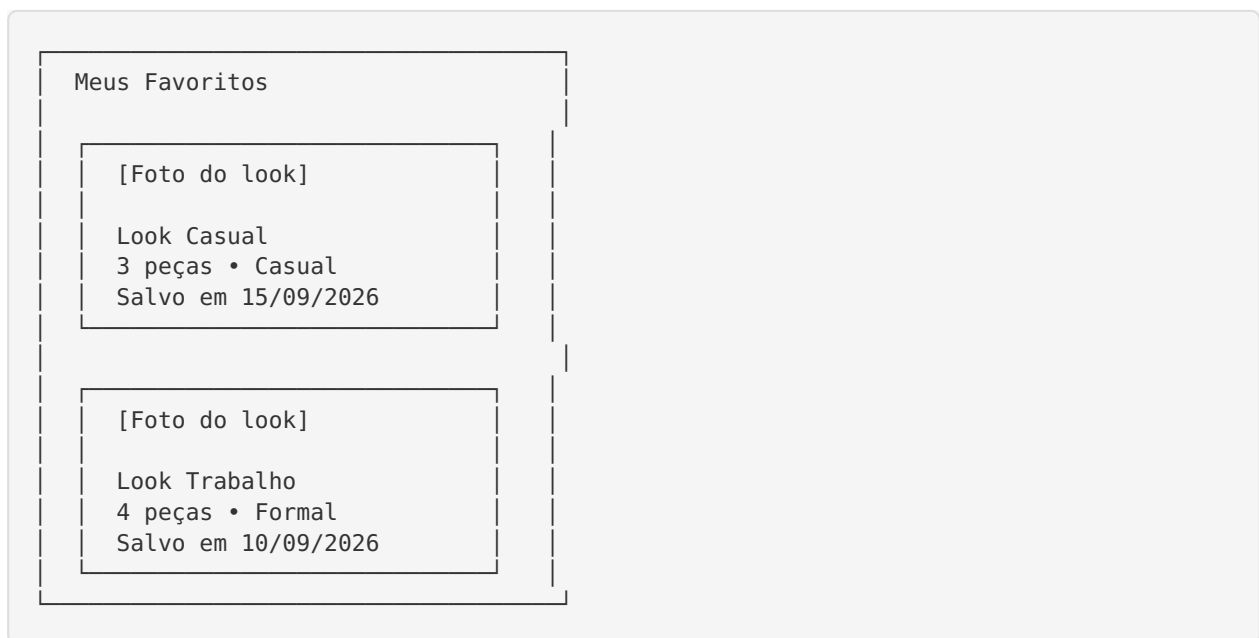
5.8 Está em Alta + Combina Comigo?

Layout:



5.9 Favoritos

Layout:



5.10 Perfil

Layout:

Perfil

[Foto do usuário]
[Nome do usuário]
[Email]

Preferências de Estilo

Casual, Streetwear
[Editar]

Estatísticas

- 45 peças no guarda-roupa
- 23 looks criados
- 12 favoritos

Configurações

- Notificações
- Privacidade
- Clima e localização
- Sobre

[Sair]

6. FLUXO DE USUÁRIO

6.1 Fluxo de Onboarding

1. Splash Screen
↓
2. Login/Cadastro
↓
3. Boas-vindas
"Bem-vindo ao STYLEWISE"
↓
4. Configuração de Perfil
 - Nome
 - Foto (opcional)
 - Preferências de estilo↓
5. Tutorial Rápido
 - "Fotografe suas roupas"
 - "A IA organiza automaticamente"
 - "Receba sugestões personalizadas"↓
6. Home

6.2 Fluxo: Adicionar Peça ao Guarda-roupa

1. Guarda-roupa → [+ Adicionar peça]
↓
2. Escolher fonte
 - Câmera
 - Galeria↓
3. Capturar/Selecionar foto
↓
4. Preview da foto
[Confirmar] [Refazer]
↓
5. IA analisa a imagem
Loading: "Analisando sua peça..."
↓
6. Resultado da análise
 - Categoria detectada
 - Cor detectada
 - Padrão detectado
 - Estilo detectado[Editar] [Salvar]
↓
7. Peça salva no guarda-roupa
Feedback: "Peça adicionada com sucesso!"

6.3 Fluxo: Análise de Look

1. Home → [Analisar meu look]
↓
 2. Capturar foto do look
 - Câmera
 - Galeria↓
 3. Preview da foto
[Confirmar] [Refazer]
↓
 4. IA analisa o look
Loading: "Analisando seu look..."
↓
 5. Resultado da análise
 - Indicadores visuais (Combinação, Cores, Estilo)
 - O que funciona
 - O que pode melhorar
 - Sugestão da IA↓
 6. Opções
 - [Ver alternativas do guarda-roupa]
 - [Salvar nos favoritos]
 - [Nova análise]
-

6.4 Fluxo: Monte Meu Look

1. Home → [Montar meu look]
 - ↓
2. Selecionar peça base
 - Escolher do guarda-roupa
 - ↓
3. Configurar parâmetros
 - Ocasiaão (dropdown)
 - Estilo (dropdown)
 - Clima (automático ou manual)
 - ↓
4. [🌟 Criar combinações]
 - ↓
5. IA gera combinações
 - Loading: "Criando combinações..."
 - ↓
6. Resultado: 3-5 looks
 - Visualização de cada look
 - Peças utilizadas
 - Explicação
 - [👍] [👎] [❤️ Salvar]
 - ↓
7. Avaliação do usuário
 - Sistema aprende preferências
 - ↓
8. Opções
 - [Gerar novas combinações]
 - [Voltar]

6.5 Fluxo: Tenho Uma Ocasiaão

1. Criar → [Tenho uma ocasião]
 - ↓
2. Descrever ocasião
 - Campo de texto livre
 - Ou selecionar pré-definida
 - Exemplo: "Jantar sábado à noite"
 - ↓
3. Configurações adicionais
 - Data/horário
 - Clima (automático)
 - ↓
4. [Gerar sugestões]
 - ↓
5. IA analisa e gera looks
 - Loading: "Criando looks para sua ocasião..."
 - ↓
6. Resultado: 3-5 looks adequados
 - Visualização
 - Peças utilizadas
 - Por que funciona para a ocasião
 - [👍] [👎] [❤️ Salvar]
 - ↓
7. Salvar look favorito (opcional)

6.6 Fluxo: Radar de Moda

1. Radar → [Tendências atuais]
↓
2. Lista de tendências
 - Filtros (categoria, status)
 - Cards de tendências↓
3. Selecionar tendência
↓
4. Detalhes da tendência
 - Imagem
 - Status
 - Gráfico de evolução
 - Fontes
 - Última atualização↓
5. Verificar compatibilidade
"Você possui X peças relacionadas"
↓
6. [Ver combinações]
↓
7. Looks gerados com peças do usuário

6.7 Fluxo: Está em Alta + Combina Comigo?

1. Radar → [Está em Alta + Combina Comigo?]
↓
 2. Sistema cruza:
 - Tendências atuais
 - Guarda-roupa do usuário
 - Perfil de estilo
 - Preferências↓
 3. Lista de tendências compatíveis
 - Tendência
 - Status
 - Peças que o usuário possui
 - Compatibilidade com estilo↓
 4. Selecionar tendência
↓
 5. Ver combinações personalizadas
-

7. DESIGN SYSTEM

7.1 Paleta de Cores

```
/* Cores Principais */
--preto-principal: #111111;
--off-white: #F7F5F2;
--champagne: #C8A96B;
--texto-principal: #171717;
--texto-secundario: #77736D;
--branco-cards: #FFFFFF;

/* Cores de Status */
--sucesso: #4CAF50;
--erro: #F44336;
--aviso: #FF9800;
--info: #2196F3;

/* Cores do Radar */
--tendencia-crescendo: #FF6B6B;
--tendencia-consolidada: #4CAF50;
--tendencia-estavel: #FFC107;
--tendencia-perdendo: #F44336;
```

7.2 Tipografia

Fonte Principal (Títulos):

- **Playfair Display** ou **Cormorant Garamond**
- Peso: 400, 600, 700
- Uso: Títulos principais, headings, elementos editoriais

Fonte Secundária (Corpo):

- **Inter** ou **DM Sans**
- Peso: 400, 500, 600
- Uso: Textos, menus, botões, informações

Hierarquia:

```
/* Títulos */
h1: 32px / 600 / Playfair Display
h2: 24px / 600 / Playfair Display
h3: 20px / 600 / Inter
h4: 18px / 600 / Inter

/* Corpo */
body: 16px / 400 / Inter
small: 14px / 400 / Inter
caption: 12px / 400 / Inter

/* Botões */
button: 16px / 600 / Inter
```

7.3 Espaçamento

```
/* Sistema de espaçamento (múltiplos de 8px) */  
--space-xs: 4px;  
--space-sm: 8px;  
--space-md: 16px;  
--space-lg: 24px;  
--space-xl: 32px;  
--space-2xl: 48px;  
--space-3xl: 64px;
```

7.4 Componentes

Botões

Botão Principal:

```
background: #111111;  
color: #FFFFFF;  
padding: 16px 32px;  
border-radius: 12px;  
font-weight: 600;  
transition: all 0.3s ease;  
  
hover:  
  background: #2A2A2A;  
  transform: translateY(-2px);
```

Botão Secundário:

```
background: #FFFFFF;  
color: #171717;  
border: 1px solid #D8D3CA;  
padding: 16px 32px;  
border-radius: 12px;  
font-weight: 600;  
transition: all 0.3s ease;  
  
hover:  
  border-color: #C8A96B;  
  background: #FAFAF8;
```

Botão com Destaque (Champagne):

```
background: #C8A96B;  
color: #FFFFFF;  
padding: 16px 32px;  
border-radius: 12px;  
font-weight: 600;  
transition: all 0.3s ease;  
  
hover:  
  background: #B89A5F;  
  transform: translateY(-2px);
```

Cards

Card Padrão:

```
background: #FFFFFF;
border-radius: 16px;
padding: 24px;
box-shadow: 0 2px 8px rgba(0, 0, 0, 0.04);
transition: all 0.3s ease;

hover:
  box-shadow: 0 4px 16px rgba(0, 0, 0, 0.08);
  transform: translateY(-4px);
```

Card de Peça (Guarda-roupa):

```
background: #FFFFFF;
border-radius: 12px;
overflow: hidden;
box-shadow: 0 2px 8px rgba(0, 0, 0, 0.04);
aspect-ratio: 3/4;

hover:
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
```

Card de Tendência (Radar):

```
background: #1A1A1A;
border-radius: 16px;
padding: 24px;
border: 1px solid #2A2A2A;
transition: all 0.3s ease;

hover:
  border-color: #C8A96B;
  box-shadow: 0 4px 16px rgba(200, 169, 107, 0.1);
```

Inputs

Input de Texto:

```
background: #FFFFFF;
border: 1px solid #D8D3CA;
border-radius: 12px;
padding: 14px 16px;
font-size: 16px;
color: #171717;
transition: all 0.3s ease;

focus:
  border-color: #C8A96B;
  outline: none;
  box-shadow: 0 0 0 3px rgba(200, 169, 107, 0.1);
```

Dropdown:

```
background: #FFFFFF;
border: 1px solid #D8D3CA;
border-radius: 12px;
padding: 14px 16px;
font-size: 16px;
color: #171717;
cursor: pointer;
transition: all 0.3s ease;

hover:
  border-color: #C8A96B;
```

Indicadores**Barra de Progresso:**

```
background: #E8E5E0;
height: 8px;
border-radius: 4px;
overflow: hidden;

.progress-fill {
  background: linear-gradient(90deg, #C8A96B, #B89A5F);
  height: 100%;
  transition: width 0.5s ease;
}
```

Badge de Status:

```
/* Crescendo */
background: rgba(255, 107, 107, 0.1);
color: #FF6B6B;
padding: 6px 12px;
border-radius: 20px;
font-size: 12px;
font-weight: 600;

/* Consolidada */
background: rgba(76, 175, 80, 0.1);
color: #4CAF50;

/* Estável */
background: rgba(255, 193, 7, 0.1);
color: #FFC107;

/* Perdendo força */
background: rgba(244, 67, 54, 0.1);
color: #F44336;
```

Navegação**Bottom Navigation:**

```

background: #FFFFFF;
border-top: 1px solid #E8E5E0;
padding: 12px 0;
position: fixed;
bottom: 0;
width: 100%;
display: flex;
justify-content: space-around;
box-shadow: 0 -2px 8px rgba(0, 0, 0, 0.04);

.nav-item {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 4px;
  color: #77736D;
  transition: all 0.3s ease;
}

.nav-item.active {
  color: #111111;
}

.nav-item.active .icon {
  color: #C8A96B;
}

```

7.5 Animações

Transições Padrão:

```
transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1);
```

Loading (Análise de IA):

```

@keyframes pulse {
  0%, 100% { opacity: 1; }
  50% { opacity: 0.5; }
}

.loading-indicator {
  animation: pulse 2s ease-in-out infinite;
}

```

Fade In:

```
@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(20px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

.fade-in {
  animation: fadeIn 0.5s ease-out;
}
```

Slide Up:

```
@keyframes slideUp {
  from {
    transform: translateY(100%);
  }
  to {
    transform: translateY(0);
  }
}

.slide-up {
  animation: slideUp 0.4s cubic-bezier(0.4, 0, 0.2, 1);
}
```

7.6 Ícones

Biblioteca: Lucide Icons ou Heroicons

Estilo: Outline (minimalista)

Tamanho padrão: 24px

Cor: Herda do texto ou #77736D

7.7 Imagens

Aspect Ratios:

- Peças do guarda-roupa: 3:4
- Cards de look: 4:5
- Tendências: 16:9
- Foto de perfil: 1:1

Border Radius:

- Imagens de peças: 12px
- Imagens de looks: 16px
- Foto de perfil: 50% (circular)

8. ARQUITETURA FRONTEND

8.1 Tecnologia

Framework: Next.js 14+ (App Router)

Por quê?

- Renderização híbrida (SSR + CSR)
 - Otimização automática de imagens
 - Roteamento baseado em arquivos
 - API Routes integradas
 - Excelente performance
 - SEO-friendly
-

8.2 Estrutura de Pastas

```
/app
├── (auth)
│   ├── login
│   │   └── page.tsx
│   ├── cadastro
│   │   └── page.tsx
│   └── layout.tsx
├── (app)
│   ├── layout.tsx
│   ├── page.tsx (Home)
│   ├── guarda-roupa
│   │   ├── page.tsx
│   │   ├── adicionar
│   │   │   └── page.tsx
│   │   └── [id]
│   │       └── page.tsx
│   ├── analisar-look
│   │   └── page.tsx
│   ├── montar-look
│   │   ├── page.tsx
│   │   └── resultado
│   │       └── page.tsx
│   ├── ocasiao
│   │   └── page.tsx
│   ├── radar
│   │   ├── page.tsx
│   │   ├── [id]
│   │   │   └── page.tsx
│   │   └── combina-comigo
│   │       └── page.tsx
│   ├── favoritos
│   │   └── page.tsx
│   ├── historico
│   │   └── page.tsx
│   └── perfil
│       └── page.tsx
├── api
│   ├── auth
│   │   ├── [...nextauth]
│   │   └── route.ts
│   ├── wardrobe
│   │   ├── route.ts
│   │   ├── [id]
│   │   └── route.ts
│   ├── analyze-look
│   │   └── route.ts
│   ├── generate-outfits
│   │   └── route.ts
│   ├── trends
│   │   └── route.ts
│   └── preferences
│       └── route.ts
└── globals.css

/components
├── ui
│   ├── Button.tsx
│   ├── Card.tsx
│   └── Input.tsx
```

```

├── Dropdown.tsx
├── Badge.tsx
├── Loading.tsx
├── ...
├── layout
│   ├── Header.tsx
│   ├── BottomNav.tsx
│   └── Container.tsx
├── wardrobe
│   ├── WardrobeGrid.tsx
│   ├── WardrobeItem.tsx
│   ├── AddItemButton.tsx
│   └── CategoryFilter.tsx
├── outfit
│   ├── OutfitCard.tsx
│   ├── OutfitVisualization.tsx
│   └── OutfitRating.tsx
├── analysis
│   ├── LookAnalysis.tsx
│   ├── ScoreIndicator.tsx
│   └── Suggestions.tsx
├── radar
│   ├── TrendCard.tsx
│   ├── TrendChart.tsx
│   └── TrendFilters.tsx
└── ...

```

/lib

```

├── api
│   ├── wardrobe.ts
│   ├── ai.ts
│   ├── trends.ts
│   └── ...
├── utils
│   ├── image.ts
│   ├── colors.ts
│   └── ...
├── hooks
│   ├── useWardrobe.ts
│   ├── usePreferences.ts
│   └── ...
├── types
│   ├── wardrobe.ts
│   ├── outfit.ts
│   ├── trend.ts
│   └── ...

```

/public

```

├── images
├── icons
└── ...

```

8.3 Bibliotecas Principais

```
{
  "dependencies": {
    "next": "^14.0.0",
    "react": "^18.0.0",
    "react-dom": "^18.0.0",
    "typescript": "^5.0.0",

    // UI
    "tailwindcss": "^3.4.0",
    "framer-motion": "^11.0.0",
    "lucide-react": "^0.300.0",

    // Formulários
    "react-hook-form": "^7.49.0",
    "zod": "^3.22.0",

    // Estado
    "zustand": "^4.4.0",

    // Requisições
    "axios": "^1.6.0",
    "swr": "^2.2.0",

    // Autenticação
    "next-auth": "^4.24.0",

    // Upload de imagens
    "react-dropzone": "^14.2.0",

    // Gráficos (Radar)
    "recharts": "^2.10.0",

    // Datas
    "date-fns": "^3.0.0"
  }
}
```

8.4 Gerenciamento de Estado

Zustand para estado global

```
// /lib/store/wardrobe.ts
import { create } from 'zustand';

interface WardrobeStore {
  items: WardrobeItem[];
  addItem: (item: WardrobeItem) => void;
  removeItem: (id: string) => void;
  updateItem: (id: string, data: Partial<WardrobeItem>) => void;
}

export const useWardrobeStore = create<WardrobeStore>((set) => ({
  items: [],
  addItem: (item) => set((state) => ({ items: [...state.items, item] })),
  removeItem: (id) => set((state) => ({
    items: state.items.filter(i => i.id !== id)
  })),
  updateItem: (id, data) => set((state) => ({
    items: state.items.map(i => i.id === id ? { ...i, ...data } : i)
  })),
}));
```

8.5 Hooks Customizados

```
// /lib/hooks/useWardrobe.ts
export function useWardrobe() {
  const { data, error, mutate } = useSWR('/api/wardrobe', fetcher);

  const addItem = async (item: WardrobeItem) => {
    await axios.post('/api/wardrobe', item);
    mutate();
  };

  return {
    items: data?.items || [],
    isLoading: !error && !data,
    error,
    addItem,
  };
}

// /lib/hooks/usePreferences.ts
export function usePreferences() {
  const { data, mutate } = useSWR('/api/preferences', fetcher);

  const updatePreference = async (key: string, value: any) => {
    await axios.patch('/api/preferences', { [key]: value });
    mutate();
  };

  return {
    preferences: data || {},
    updatePreference,
  };
}
```

9. ARQUITETURA BACKEND

9.1 Tecnologia

Framework: Next.js API Routes + Prisma ORM

Por quê?

- Integração perfeita com frontend
 - TypeScript end-to-end
 - Prisma oferece type-safety
 - Fácil deploy
 - Escalável
-

9.2 Estrutura de APIs

```
/app/api
├── auth
│   └── [...nextauth]
│       └── route.ts
├── wardrobe
│   ├── route.ts (GET, POST)
│   ├── [id]
│   │   └── route.ts (GET, PATCH, DELETE)
│   └── analyze
│       └── route.ts (POST)
├── outfits
│   ├── generate
│   │   └── route.ts (POST)
│   ├── favorites
│   │   └── route.ts (GET, POST)
│   └── history
│       └── route.ts (GET)
├── analyze
│   └── look
│       └── route.ts (POST)
├── trends
│   ├── route.ts (GET)
│   ├── [id]
│   │   └── route.ts (GET)
│   └── match
│       └── route.ts (GET)
├── preferences
│   └── route.ts (GET, PATCH)
├── weather
│   └── route.ts (GET)
└── upload
    └── route.ts (POST)
```

9.3 Exemplos de Endpoints

POST /api/wardrobe

Adicionar peça ao guarda-roupa

```
// Request
{
  "imageUrl": "https://i.ebayimg.com/images/g/l6EAA0SwPJdhksad/s-l1200.jpg",
  "category": "camiseta",
  "color": "preta",
  "pattern": "lisa",
  "style": "casual"
}

// Response
{
  "id": "uuid",
  "userId": "uuid",
  "imageUrl": "https://i5.walmartimages.com/seo/RICHCOIN-Short-Sleeve-T-shirt-for-Men-Crew-Neck-Summer-Top-Casual-Loose-Plain-Black-Tee-Shirt-L_13d5f2b9-11dc-4730-b154-2279c62bf302.1aec7ca9536a9fec06c4a8058303aba5.jpeg",
  "category": "camiseta",
  "color": "preta",
  "pattern": "lisa",
  "style": "casual",
  "createdAt": "2026-09-21T..."
}
```

POST /api/wardrobe/analyze

Analisar imagem de peça com IA

```
// Request
{
  "imageUrl": "https://backlinko.com/_next/image?url=https%3A%2F%2Fapi.backlinko.com%2Fapp%2Fuploads%2F2025%2F03%2Furl-parameters-featured-image-1.png&w=3840&q=75"
}

// Response
{
  "category": "camiseta",
  "color": "preta",
  "pattern": "lisa",
  "style": "casual",
  "confidence": 0.95
}
```

POST /api/analyze/look

Analisar look completo

```
// Request
{
  "imageUrl": "https://labs.scale.com/_next/image?url=https%3A%2F%2Fcdn.sanity.io%2Fimages%2F5uhyv5jy%2Fproduction%2F9d1705783e7d7596f05dfba2d-f321775b0865165-1323x857.png%3Fw%3D1200%26fit%3Dcrop%26auto%3Dformat&w=3840&q=75"
}

// Response
{
  "scores": {
    "combination": 8,
    "colors": 10,
    "style": 7
  },
  "whatWorks": [
    "Cores neutras equilibradas",
    "Tênis branco cria contraste"
  ],
  "improvements": [
    "Adicionar um acessório"
  ],
  "suggestion": "Experimente adicionar um relógio ou pulseira",
  "detectedItems": [
    { "category": "camiseta", "color": "preta" },
    { "category": "calça", "color": "jeans" },
    { "category": "tênis", "color": "branco" }
  ]
}
```

POST /api/outfits/generate

Gerar combinações de look

```
// Request
{
  "baseItemId": "uuid",
  "occasion": "casual",
  "style": "streetwear",
  "weather": {
    "temp": 25,
    "condition": "sunny"
  }
}

// Response
{
  "outfits": [
    {
      "id": "uuid",
      "items": [
        { "id": "uuid", "category": "camiseta", "imageUrl": "..." },
        { "id": "uuid", "category": "calça", "imageUrl": "..." },
        { "id": "uuid", "category": "tênis", "imageUrl": "..." }
      ],
      "explanation": "Visual casual equilibrado com contraste no calçado",
      "occasion": "casual",
      "style": "streetwear"
    },
    // ... mais 2-4 outfits
  ]
}
```

GET /api/trends

Listar tendências

```
// Query params: ?category=jaquetas&status=crescendo

// Response
{
  "trends": [
    {
      "id": "uuid",
      "name": "Jaqueta Oversized",
      "category": "jaquetas",
      "status": "crescendo",
      "imageUrl": "https://i.pinimg.com/736x/a0/97/fa/a097faaaa2f34b5346f2ade80b-deb5cd.jpg",
      "evolution": [
        { "date": "2026-09-01", "score": 65 },
        { "date": "2026-09-15", "score": 78 },
        { "date": "2026-09-21", "score": 85 }
      ],
      "sources": ["Google Trends", "Fashion API"],
      "lastUpdated": "2026-09-21T..."
    },
    // ...
  ]
}
```

GET /api/trends/match

Tendências que combinam com o guarda-roupa

```
// Response
{
  "matches": [
    {
      "trend": {
        "id": "uuid",
        "name": "Jaqueta Oversized",
        "status": "crescendo",
        "imageUrl": "..."
      },
      "userItems": [
        { "id": "uuid", "category": "jaqueta", "imageUrl": "..." },
        { "id": "uuid", "category": "jaqueta", "imageUrl": "..." }
      ],
      "matchScore": 0.85,
      "styleCompatibility": true
    },
    // ...
  ]
}
```

10. BANCO DE DADOS

10.1 Tecnologia

PostgreSQL (hospedado pela Abacus.AI)

ORM: Prisma

Por quê?

- Relacional e robusto
 - Prisma oferece type-safety
 - Migrações automáticas
 - Excelente DX (Developer Experience)
-

10.2 Schema do Banco de Dados

```
// schema.prisma

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// =====
// USUÁRIOS
// =====

model User {
  id          String    @id @default(uuid())
  email       String    @unique
  name        String?
  password    String
  avatar      String?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  // Relações
  wardrobeItems WardrobeItem[]
  outfits        Outfit[]
  favorites       Favorite[]
  preferences     UserPreference?
  history         History[]
  ratings         OutfitRating[]

  @@map("users")
}

// =====
// PREFERÊNCIAS DO USUÁRIO
// =====

model UserPreference {
  id          String    @id @default(uuid())
  userId      String    @unique
  user        User      @relation(fields: [userId], references: [id], onDelete: Cascade)

  // Preferências de estilo
  styles      String[] // ["casual", "streetwear", "elegante"]

  // Preferências de cores
  favoriteColors String[]
  avoidColors    String[]

  // Ocasões frequentes
  occasions      String[]

  // Localização (para clima)
  location       String?

  createdAt      DateTime @default(now())
  updatedAt      DateTime @updatedAt

  @@map("user_preferences")
}
```

```
// =====
// GUARDA-ROUPA
// =====

model WardrobeItem {
  id          String    @id @default(uuid())
  userId      String
  user        User      @relation(fields: [userId], references: [id], onDelete: Cascade)

  // Imagem
  imageUrl    String

  // Informações da peça
  category    String    // "camiseta", "calça", "tênis", etc.
  color       String    // "preta", "azul", "branca", etc.
  pattern     String?   // "lisa", "listrada", "estampada", etc.
  style       String?   // "casual", "formal", "esportivo", etc.

  // Metadados
  brand       String?
  season      String?   // "verão", "inverno", "meia-estação"

  // Análise da IA
  aiAnalysis  Json?

  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  // Relações
  outfitItems OutfitItem[]

  @@index([userId, category])
  @@map("wardrobe_items")
}

// =====
// LOOKS/OUTFITS
// =====

model Outfit {
  id          String    @id @default(uuid())
  userId      String
  user        User      @relation(fields: [userId], references: [id], onDelete: Cascade)

  // Informações do look
  name        String?
  occasion    String?   // "casual", "trabalho", "festa", etc.
  style       String?   // "casual", "formal", etc.

  // Explicação da IA
  explanation String?

  // Metadados
  weather     Json?     // { temp: 25, condition: "sunny" }

  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  // Relações
  items       OutfitItem[]
  favorites   Favorite[]
}
```

```

    ratings      OutfitRating[]

    @@index([userId])
    @@map("outfits")
  }

  model OutfitItem {
    id            String      @id @default(uuid())
    outfitId      String
    outfit        Outfit      @relation(fields: [outfitId], references: [id],
    onDelete: Cascade)
    wardrobeItemId String
    wardrobeItem  WardrobeItem @relation(fields: [wardrobeItemId], references: [id],
    onDelete: Cascade)

    // Ordem no look (1 = primeira peça, 2 = segunda, etc.)
    order         Int

    @@unique([outfitId, wardrobeItemId])
    @@map("outfit_items")
  }

  // =====
  // FAVORITOS
  // =====

  model Favorite {
    id            String      @id @default(uuid())
    userId        String
    user          User        @relation(fields: [userId], references: [id], onDelete: Cascade)
    outfitId      String
    outfit        Outfit      @relation(fields: [outfitId], references: [id], onDelete: Cascade)

    createdAt    DateTime @default(now())

    @@unique([userId, outfitId])
    @@map("favorites")
  }

  // =====
  // AVALIAÇÕES (👍/👎)
  // =====

  model OutfitRating {
    id            String      @id @default(uuid())
    userId        String
    user          User        @relation(fields: [userId], references: [id], onDelete: Cascade)
    outfitId      String
    outfit        Outfit      @relation(fields: [outfitId], references: [id], onDelete: Cascade)

    // Avaliação: true = 👍, false = 👎
    liked         Boolean

    createdAt    DateTime @default(now())

    @@unique([userId, outfitId])
    @@map("outfit_ratings")
  }

  // =====
  // HISTÓRICO

```

```
// =====

model History {
  id          String    @id @default(uuid())
  userId      String
  user        User      @relation(fields: [userId], references: [id], onDelete: Cascade)

  // Tipo de ação
  type        String    // "look_analyzed", "outfit_generated", "item_added", etc.

  // Dados da ação
  data        Json

  createdAt   DateTime @default(now())

  @@index([userId, createdAt])
  @@map("history")
}

// =====
// RADAR DE MODA
// =====

model Trend {
  id          String    @id @default(uuid())

  // Informações da tendência
  name        String
  category     String    // "jaquetas", "calçados", "acessórios", etc.
  description  String?
  imageUrl     String?

  // Status
  status       String    // "crescendo", "consolidada", "estavel", "perdendo"

  // Evolução (dados históricos)
  evolution    Json      // [{ date: "2026-09-01", score: 65 }, ...]

  // Fontes
  sources      String[]  // ["Google Trends", "Fashion API"]

  // Metadados
  lastUpdated  DateTime @default(now())
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt

  @@index([category, status])
  @@map("trends")
}
```

10.3 Relacionamentos

```
User (1) — (N) WardrobeItem
User (1) — (N) Outfit
User (1) — (N) Favorite
User (1) — (1) UserPreference
User (1) — (N) History
User (1) — (N) OutfitRating

Outfit (1) — (N) OutfitItem
WardrobeItem (1) — (N) OutfitItem

Outfit (1) — (N) Favorite
Outfit (1) — (N) OutfitRating
```

11. SISTEMA DE IA

11.1 Análise de Imagens

Tecnologia: Abacus.AI Vision API

Funcionalidades:

1. Identificação de Peças

- Categoria (camiseta, calça, tênis, etc.)
- Cor dominante
- Padrão (lisa, listrada, estampada)
- Estilo (casual, formal, esportivo)

2. Análise de Look Completo

- Identificar todas as peças
- Avaliar combinação de cores
- Avaliar harmonia do conjunto
- Detectar pontos de melhoria

11.2 Sistema de Recomendação

Algoritmo:

```
function generateOutfits(
  baseItem: WardrobeItem,
  wardrobe: WardrobeItem[],
  preferences: UserPreference,
  occasion: string,
  weather: Weather
): Outfit[] {

  // 1. Filtrar peças compatíveis
  const compatibleItems = wardrobe.filter(item => {
    return (
      isColorCompatible(baseItem.color, item.color) &&
      isStyleCompatible(baseItem.style, item.style) &&
      isOccasionAppropriate(item, occasion) &&
      isWeatherAppropriate(item, weather)
    );
  });

  // 2. Gerar combinações
  const combinations = generateCombinations(baseItem, compatibleItems);

  // 3. Pontuar combinações
  const scored = combinations.map(combo => ({
    ...combo,
    score: scoreOutfit(combo, preferences)
  }));

  // 4. Ordenar por pontuação
  const sorted = scored.sort((a, b) => b.score - a.score);

  // 5. Retornar top 3-5
  return sorted.slice(0, 5);
}
```

11.3 Sistema de Aprendizado

Estratégia:

1. Coleta de Feedback

- Cada outfit recebe 👍 ou 👎
- Armazenar no banco de dados

2. Análise de Padrões

```
``typescript
```

```
function analyzePreferences(ratings: OutfitRating[]): Preferences {
  const liked = ratings.filter(r => r.liked);
  const disliked = ratings.filter(r => !r.liked);

  return {
    favoriteColors: extractColors(liked),
    avoidColors: extractColors(disliked),
    favoriteStyles: extractStyles(liked),
    avoidStyles: extractStyles(disliked),
    // ...
  };
}
```

```
}
...
```

3. Ajuste de Recomendações

- Aumentar peso de características preferidas
- Diminuir peso de características rejeitadas
- Atualização gradual (não abrupta)

11.4 Análise de Cores

Regras de Combinação:

```
const colorRules = {
  // Cores neutras combinam com tudo
  neutral: ['preto', 'branco', 'cinza', 'bege', 'marrom'],

  // Cores complementares
  complementary: {
    'azul': ['laranja', 'amarelo'],
    'vermelho': ['verde', 'azul'],
    'amarelo': ['roxo', 'azul'],
    // ...
  },

  // Cores análogas
  analogous: {
    'azul': ['verde', 'roxo'],
    'vermelho': ['laranja', 'rosa'],
    // ...
  }
};

function isColorCompatible(color1: string, color2: string): boolean {
  // Neutras sempre combinam
  if (isNeutral(color1) || isNeutral(color2)) return true;

  // Mesma cor em tons diferentes
  if (isSameColorFamily(color1, color2)) return true;

  // Cores complementares
  if (isComplementary(color1, color2)) return true;

  // Cores análogas
  if (isAnalogous(color1, color2)) return true;

  return false;
}
```

12. SEGURANÇA E PRIVACIDADE

12.1 Autenticação

NextAuth.js com estratégia de credenciais

```
// /app/api/auth/[...nextauth]/route.ts
import NextAuth from 'next-auth';
import CredentialsProvider from 'next-auth/providers/credentials';
import { compare } from 'bcrypt';
import { prisma } from '@lib/prisma';

export const authOptions = {
  providers: [
    CredentialsProvider({
      credentials: {
        email: { type: 'email' },
        password: { type: 'password' }
      },
      async authorize(credentials) {
        const user = await prisma.user.findUnique({
          where: { email: credentials.email }
        });

        if (!user) return null;

        const isValid = await compare(credentials.password, user.password);

        if (!isValid) return null;

        return {
          id: user.id,
          email: user.email,
          name: user.name
        };
      }
    })
  ],
  session: {
    strategy: 'jwt'
  },
  pages: {
    signIn: '/login'
  }
};

const handler = NextAuth(authOptions);
export { handler as GET, handler as POST };
```

12.2 Proteção de Rotas

```
// middleware.ts
import { withAuth } from 'next-auth/middleware';

export default withAuth({
  pages: {
    signIn: '/login'
  }
});

export const config = {
  matcher: [
    '/guarda-roupa/:path*',
    '/analisar-look/:path*',
    '/montar-look/:path*',
    '/radar/:path*',
    '/favoritos/:path*',
    '/perfil/:path*'
  ]
};
```

12.3 Privacidade das Imagens

Regras:

1. Armazenamento Privado

- Imagens armazenadas no Cloud Storage da Abacus.AI
- URLs assinadas com expiração
- Acesso restrito ao proprietário

2. Não Compartilhamento

- Fotos nunca são compartilhadas com terceiros
- Não são usadas para treinamento de IA sem consentimento
- Podem ser deletadas a qualquer momento

3. Política de Retenção

- Usuário pode deletar peças e looks
- Deleção permanente após 30 dias
- Backup criptografado

12.4 LGPD / GDPR

Conformidade:

1. Consentimento

- Termos de uso claros
- Política de privacidade acessível
- Opt-in para funcionalidades opcionais

2. Direitos do Usuário

- Acesso aos dados

- Correção de dados
- Deleção de dados
- Portabilidade de dados
- Revogação de consentimento

3. Segurança

- Senhas com hash (bcrypt)
- Comunicação HTTPS
- Tokens JWT seguros
- Rate limiting nas APIs

13. ESTRATÉGIA DE MVP

13.1 Funcionalidades do MVP

FASE 1 — MVP CORE (4-6 semanas)

✓ Essenciais:

1. Autenticação

- Cadastro
- Login
- Recuperação de senha

2. Guarda-roupa Digital

- Adicionar peça (foto + IA)
- Visualizar peças
- Editar peça
- Deletar peça
- Filtros básicos (categoria)

3. Análise de Look

- Fotografar look
- Análise da IA
- Sugestões básicas

4. Monte Meu Look

- Selecionar peça base
- Gerar 3 combinações
- Explicação de cada look
- Avaliação 👍/👎

5. Favoritos

- Salvar looks
- Visualizar favoritos

6. Perfil Básico

- Dados pessoais
 - Preferências de estilo (manual)
-

FASE 2 — APRENDIZADO E PERSONALIZAÇÃO (2-3 semanas)

✓ Importantes:

1. Sistema de Aprendizado

- Análise de avaliações 👍/👎
- Ajuste de recomendações
- Preferências automáticas

2. Tenho Uma Ocasião

- Descrição da ocasião
- Sugestões personalizadas

3. Histórico

- Looks analisados
- Combinações geradas

4. Clima

- Integração com API de clima
- Sugestões baseadas no tempo

FASE 3 — RADAR DE MODA (3-4 semanas)

✓ Diferenciais:

1. Radar de Moda

- Lista de tendências
- Detalhes da tendência
- Gráficos de evolução
- Fontes e rastreabilidade

2. Está em Alta + Combina Comigo?

- Cruzamento de tendências com guarda-roupa
- Sugestões personalizadas

3. Tendências que Combinam

- Filtros avançados
- Compatibilidade com estilo

13.2 Funcionalidades Pós-MVP

FASE 4 — EXPANSÃO

- Visualização 3D de looks
- Compartilhamento de looks
- Comunidade/social
- Planejamento semanal de looks
- Sugestões de compras inteligentes

- Integração com e-commerce (afiliados)
 - Notificações push
 - Modo offline
 - Exportar guarda-roupa (PDF)
-

13.3 Métricas de Sucesso do MVP

KPIs Principais:

1. Engajamento

- Peças adicionadas por usuário
- Looks gerados por semana
- Taxa de retorno (DAU/MAU)

2. Satisfação

- Avaliações positivas (👍) vs negativas (👎)
- NPS (Net Promoter Score)
- Tempo médio na plataforma

3. Retenção

- Usuários ativos após 7 dias
- Usuários ativos após 30 dias
- Churn rate

4. Funcionalidades

- Uso do “Monte Meu Look”
 - Uso do “Análise de Look”
 - Uso do “Radar de Moda”
-

14. TECNOLOGIAS

14.1 Stack Completo

```
FRONTEND
├─ Next.js 14+ (App Router)
├─ React 18+
├─ TypeScript
├─ Tailwind CSS
├─ Framer Motion (animações)
├─ Lucide Icons

BACKEND
├─ Next.js API Routes
├─ Prisma ORM
├─ PostgreSQL (Abacus.AI)

IA
├─ Abacus.AI Vision API (análise de imagens)
├─ Abacus.AI Text Generation (explicações)

ARMAZENAMENTO
├─ Abacus.AI Cloud Storage (imagens)

AUTENTICAÇÃO
├─ NextAuth.js

DEPLOY
├─ Abacus.AI Platform

FERRAMENTAS
├─ Git
├─ ESLint
├─ Prettier
├─ Husky (pre-commit hooks)
```

14.2 Justificativas das Tecnologias

Next.js 14+

Por quê?

- Framework React moderno e completo
- App Router oferece melhor performance
- Server Components reduzem bundle size
- API Routes integradas (backend + frontend)
- Otimização automática de imagens
- SEO-friendly
- Excelente DX

Alternativas:

- Remix
 - SvelteKit
 - Nuxt.js (Vue)
-

TypeScript

Por quê?

- Type-safety end-to-end
- Menos bugs em produção
- Melhor autocomplete
- Refatoração mais segura
- Documentação implícita

Alternativas:

- JavaScript puro (não recomendado)
-

Tailwind CSS

Por quê?

- Utility-first (rápido desenvolvimento)
- Design system consistente
- Purge CSS automático (bundle pequeno)
- Responsividade fácil
- Customização total

Alternativas:

- CSS Modules
 - Styled Components
 - Emotion
-

Prisma ORM

Por quê?

- Type-safety com TypeScript
- Migrações automáticas
- Excelente DX
- Query builder intuitivo
- Suporte a PostgreSQL

Alternativas:

- Drizzle ORM
 - TypeORM
 - Sequelize
-

PostgreSQL

Por quê?

- Banco relacional robusto
- Suporte a JSON (flexibilidade)
- Escalável
- Open-source
- Hospedado pela Abacus.AI

Alternativas:

- MySQL
 - MongoDB (NoSQL)
 - Supabase
-

Abacus.AI Vision API**Por quê?**

- Análise de imagens precisa
- Identificação de roupas, cores, padrões
- Integração nativa com a plataforma
- Sem necessidade de treinar modelos

Alternativas:

- Google Cloud Vision
 - AWS Rekognition
 - Azure Computer Vision
-

NextAuth.js**Por quê?**

- Integração perfeita com Next.js
- Suporte a múltiplos providers
- Seguro por padrão
- Fácil configuração

Alternativas:

- Clerk
 - Auth0
 - Firebase Auth
-

Framer Motion**Por quê?**

- Animações declarativas
- Performance otimizada
- Fácil de usar
- Gestos e interações

Alternativas:

- React Spring
 - GSAP
 - CSS Animations
-

14.3 APIs Externas

Clima

API: OpenWeatherMap ou WeatherAPI

Por quê?

- Dados climáticos precisos
- Gratuito até certo limite
- Fácil integração

Uso:

```
const weather = await fetch(
  `https://api.openweathermap.org/data/2.5/weather?q=${city}&appid=${API_KEY}`
);
```

Radar de Moda (MVP)

Estratégia para MVP:

Para o MVP, o Radar de Moda utilizará **dados demonstrativos/simulados** com estrutura preparada para integração futura.

Fontes Futuras:

- Google Trends API
- Fashion APIs (The Yes, Lyst, etc.)
- Web scraping ético de sites de moda
- Redes sociais (Instagram, Pinterest)

Implementação MVP:

```
// Dados simulados para demonstração
const mockTrends = [
  {
    id: '1',
    name: 'Jaqueta Oversized',
    category: 'jaquetas',
    status: 'crescendo',
    evolution: [
      { date: '2026-09-01', score: 65 },
      { date: '2026-09-15', score: 78 },
      { date: '2026-09-21', score: 85 }
    ],
    sources: ['Demonstração'],
    lastUpdated: new Date()
  },
  // ...
];
```

Integração Futura:

```
// Estrutura preparada para APIs reais
async function fetchTrends() {
  // Google Trends
  const googleTrends = await fetchGoogleTrends();

  // Fashion APIs
  const fashionData = await fetchFashionAPIs();

  // Processar e combinar dados
  const trends = processTrends(googleTrends, fashionData);

  // Salvar no banco
  await saveTrends(trends);

  return trends;
}
```

15. PRÓXIMOS PASSOS

15.1 Implementação

1. Setup do Projeto

- Criar projeto Next.js
- Configurar TypeScript
- Configurar Tailwind CSS
- Configurar Prisma

2. Design System

- Criar componentes UI base
- Implementar paleta de cores
- Configurar tipografia
- Criar animações

3. Autenticação

- Implementar NextAuth
- Criar telas de login/cadastro
- Proteger rotas

4. Guarda-roupa Digital

- Upload de imagens
- Integração com IA
- CRUD de peças
- Filtros e busca

5. Análise de Look

- Captura de foto
- Análise com IA
- Exibição de resultados

6. Monte Meu Look

- Seleção de peça
- Algoritmo de recomendação

- Geração de combinações
- Sistema de avaliação

7. Radar de Moda

- Estrutura de dados
- Interface do Radar
- Cruzamento com guarda-roupa

8. Testes e Deploy

- Testes unitários
- Testes de integração
- Deploy na Abacus.AI

15.2 Cronograma Estimado

SEMANA 1-2: Setup + Design System + Autenticação
 SEMANA 3-4: Guarda-roupa Digital
 SEMANA 5: Análise de Look
 SEMANA 6-7: Monte Meu Look + Sistema de Aprendizado
 SEMANA 8: Favoritos + Histórico + Perfil
 SEMANA 9-10: Radar de Moda (estrutura + dados demonstrativos)
 SEMANA 11: Testes + Ajustes
 SEMANA 12: Deploy + Documentação

Total: ~12 semanas para MVP completo

16. CONCLUSÃO

STYLEWISE é um aplicativo de moda com IA que combina:

- ✓ **Tecnologia de ponta** (IA, visão computacional)
- ✓ **Design premium** (revista de moda + tecnologia)
- ✓ **Foco no usuário** (prioriza o guarda-roupa existente)
- ✓ **Aprendizado contínuo** (IA que melhora com o tempo)
- ✓ **Diferencial competitivo** (Radar de Moda inteligente)

O aplicativo está arquitetado para ser:

- **Escalável** (arquitetura moderna)
- **Seguro** (privacidade das fotos)
- **Performático** (Next.js + otimizações)
- **Intuitivo** (UX/UI premium)
- **Inteligente** (IA que aprende)

Pronto para começar a implementação! 🚀